Assignment-13

2303A51666

Sony D

Batch-23

Task Description #1 (Refactoring – Removing Code Duplication)

• Task: Use AI to refactor a given Python script that contains multiple repeated code blocks.

• Instructions:

o Prompt AI to identify duplicate logic and replace it with functions or classes.

o Ensure the refactored code maintains the same output.

o Add docstrings to all functions.

• Sample Legacy Code:

Legacy script with repeated logic

print("Area of Rectangle:", 5 * 10)

print("Perimeter of Rectangle:", 2 * (5 + 10))

print("Area of Rectangle:", 7 * 12)

print("Perimeter of Rectangle:", 2 * (7 + 12))

print("Area of Rectangle:", 10 * 15)

print("Perimeter of Rectangle:", 2 * (10 + 15))

• Expected Output:

o Refactored code with a reusable function and no duplication. Well documented code

Ass-13.py > ...

```python
"""
# Legacy script with repeated logic
print("Area of Rectangle:", 5 * 10)
print("Perimeter of Rectangle:", 2 * (5 + 10))
print("Area of Rectangle:", 7 * 12)
print("Perimeter of Rectangle:", 2 * (7 + 12))
print("Area of Rectangle:", 10 * 15)
print("Perimeter of Rectangle:", 2 * (10 + 15)) """
#refactor the above code with a reusable function without any duplication and Well documented code
def rectangle_details(length, width):
    """
    Return area and perimeter of a rectangle.
    Args:
        length (int | float): Rectangle length
        width (int | float): Rectangle width
    Returns:
        tuple: (area, perimeter)
    """
    area = length * width
    perimeter = 2 * (length + width)
    return area, perimeter
# Rectangle inputs
rectangles = [(5, 10), (7, 12), (10, 15)]
# Process rectangles
for l, w in rectangles:
    area, perimeter = rectangle_details(l, w)
    print(f"Area of Rectangle: {area}")
    print(f"Perimeter of Rectangle: {perimeter}")
```

```
PS C:\Users\SONY REDDY\OneDrive\Desktop\AI ASSISTANT CODING> & "C:/Users/SONY REDDY/AppData/Local/Programs/Python/Python313/python.exe" "c:/Users
op/AI ASSISTANT CODING/Ass-13.py"
Area of Rectangle: 50
Perimeter of Rectangle: 30
Area of Rectangle: 50
Perimeter of Rectangle: 30
Area of Rectangle: 84
Perimeter of Rectangle: 38
Area of Rectangle: 150
Perimeter of Rectangle: 50
```

Task Description #2 (Refactoring – Extracting Reusable Functions)

• Task: Use AI to refactor a legacy script where multiple calculations are embedded directly inside the main code block.

• Instructions:

o Identify repeated or related logic and extract it into reusable functions.

o Ensure the refactored code is modular, easy to read, and documented with docstrings.

• Sample Legacy Code:

```
# Legacy script with inline repeated logic

price = 250

tax = price * 0.18

total = price + tax

print("Total Price:", total)

price = 500

tax = price * 0.18

total = price + tax

print("Total Price:", total)
```

• Expected Output:

o Code with a function calculate_total(price) that can be reused for multiple price inputs. Well documented code

```python
# Legacy script with inline repeated logic
price = 250
tax = price * 0.18
total = price + tax
print("Total Price:", total)
price = 500
tax = price * 0.18
total = price + tax
print("Total Price:", total)
#Refactor the given Python code by extracting the repeated tax and total price calculation into a reusable function named calculate_total(price).
# Ensure the code remains readable, modular, and produces the same output with proper docstrings.
def calculate_total(price):
    """
    Calculate total price including tax.
    Args:
        price (int | float): The original price of the item.
    Returns:
        float: Total price after adding tax.
    """
    tax = price * 0.18
    total = price + tax
    return total
# Prices to calculate total for
prices = [250, 500]
# Calculate and print total prices
for price in prices:
    total = calculate_total(price)
    print("Total Price:", total)
```

```
PS C:\Users\SONY REDDY\OneDrive\Desktop\AI ASSISTANT CODING> & "C:/Users/SONY REDDY/AppData/Local/Programs/Python/Python313/python.exe" "c:/Users/SONY REDDY/OneDrive/Desktop/AI ASSISTANT CODING/Ass-13.py"
Total Price: 295.0
Total Price: 590.0
Total Price: 295.0
Total Price: 590.0
```

Task Description #3: Refactoring Using Classes and Methods (Eliminating Redundant Conditional Logic)

Refactor a Python script that contains repeated if–elif–else grading logic by implementing a structured, object-oriented solution using a class and a method.

Problem Statement

The given script contains duplicated conditional statements used to assign grades based on student marks. This redundancy violates clean code principles and reduces maintainability.

You are required to refactor the script using a class-based design to improve modularity, reusability, and readability while preserving the original grading

logic.

Mandatory Implementation Requirements

1. Class Name: GradeCalculator

2. Method Name: calculate_grade(self, marks)

3. The method must:

o Accept marks as a parameter.

o Return the corresponding grade as a string.

o The grading logic must strictly follow the conditions below:

▪ Marks ≥ 90 and ≤ 100 → "Grade A"

▪ Marks ≥ 80 → "Grade B"

▪ Marks ≥ 70 → "Grade C"

▪ Marks ≥ 40 → "Grade D"

▪ Marks ≥ 0 → "Fail"

Note: Assume marks are within the valid range of 0 to 100.

4. Include proper docstrings for:

o The class

o The method (with parameter and return descriptions)

5. The method must be reusable and called multiple times without rewriting conditional logic.

• Given code:

```
marks = 85
if marks >= 90:
print("Grade A")
elif marks >= 75:
print("Grade B")
else:
print("Grade C")
marks = 72
if marks >= 90:
print("Grade A")
elif marks >= 75:
print("Grade B")
else:
print("Grade C")
```

Expected Output:

• Define a class named GradeCalculator.

• Implement a method calculate_grade(self, marks) inside the class.

• Create an object of the class.

• Call the method for different student marks.

• Print the returned grade values.

```python
"""
marks = 85
if marks >= 90:
    print("Grade A")
elif marks >= 75:
    print("Grade B")
else:
    print("Grade C")
marks = 72
if marks >= 90:
    print("Grade A")
elif marks >= 75:
    print("Grade B")
else:
    print("Grade C") """
#Refactor the code by moving the repeated grading conditions into a class GradeCalculator with
# a method calculate_grade(self, marks), add docstrings, create an object, and call the method for
# different marks while keeping the same output.
"""
Grade Calculation Module
This module provides a class-based implementation to calculate
student grades using reusable and structured logic.
"""
class GradeCalculator:
    """
    A class to calculate student grades based on marks.
    Methods:
        calculate_grade(self, marks):
            Returns the grade corresponding to the given marks.
    """
    def calculate_grade(self, marks):
        """
        Determine the grade for the given marks.
        Parameters:
            marks (int or float): Student marks between 0 and 100.
```

Ass-13.py > ...

```python
class GradeCalculator:
    def calculate_grade(self, marks):

        Determine the grade for the given marks.
        Parameters:
            marks (int or float): Student marks between 0 and 100.
        Returns:
            str: The grade corresponding to the marks.
        """
        if 90 <= marks <= 100:
            return "Grade A"
        elif marks >= 80:
            return "Grade B"
        elif marks >= 70:
            return "Grade C"
        elif marks >= 40:
            return "Grade D"
        elif marks >= 0:
            return "Fail"
# Create object of GradeCalculator
grade_calculator = GradeCalculator()
# Call method multiple times
marks1 = 85
marks2 = 72
marks3 = 35
print(grade_calculator.calculate_grade(marks1))
print(grade_calculator.calculate_grade(marks2))
print(grade_calculator.calculate_grade(marks3))
```

```
PS C:\Users\SONY REDDY\OneDrive\Desktop\AI ASSISTANT CODING> & "C:/Users/SONY REDDY/AppData/Local/Programs/Python/Python313/python.exe" "c:/Users
op/AI ASSISTANT CODING/Ass-13.py"
Grade B
Grade C
Fail
```

Task Description #4 (Refactoring – Converting Procedural Code to Functions)

• Task: Use AI to refactor procedural input–processing logic into functions.

Instructions:

o Identify input, processing, and output sections.

o Convert each into a separate function.

o Improve code readability without changing behavior.

• Sample Legacy Code:

num = int(input("Enter number: "))

square = num * num

print("Square:", square)

• Expected Output:

o Modular code using functions like get_input(), calculate_square(), and

display_result().

```
Ass-13.py > ...
"""
num = int(input("Enter number: "))
square = num * num
print("Square:", square) """
#Refactor the code by separating input, calculation, and output into functions named get_input(),
#calculate_square(num), and display_result(square) without changing the behavior.
def get_input():
    """
    Get an integer input from the user.
    Returns:
        int: Number entered by the user
    """
    return int(input("Enter number: "))
def calculate_square(num):
    """
    Calculate the square of a number.
    Args:
        num (int): Input number
    Returns:
        int: Square of the number
    """
    return num * num
def display_result(square):
    """
    Display the square result.
    Args:
        square (int): Square value to display
    """
    print("Square:", square)
number = get_input()
square = calculate_square(number)
display_result(square)
```

```
PS C:\Users\SONY REDDY\OneDrive\Desktop\AI ASSISTANT CODING> & "C:/Users/SONY REDDY/AppData/Local/Programs/Python/Python313/python.exe" "c:/Users/SONY
op/AI ASSISTANT CODING/Ass-13.py"
Enter number: 45
Square: 2025
```

Task 5 (Refactoring Procedural Code into OOP Design)

• Task: Use AI to refactor procedural code into a class-based design.Focus Areas:

o Object-Oriented principles

o Encapsulation

Legacy Code:

salary = 50000

tax = salary * 0.2

net = salary - tax

print(net)

Expected Outcome:

o A class like EmployeeSalaryCalculator with methods and attributes.

```python
salary = 50000
tax = salary * 0.2
net = salary - tax
print(net)
#Refactor the procedural salary code into a class EmployeeSalaryCalculator by creating methods for tax calculation and
# net salary calculation, add proper docstrings, and keep the final output unchanged.
"""
Employee Salary Calculation Module
This module provides a class-based implementation
to calculate tax and net salary using encapsulation.
"""
class EmployeeSalaryCalculator:
    """
    A class to calculate tax and net salary for an employee.
    Attributes:
        salary (float): The gross salary of the employee.
    """
    def __init__(self, salary):
        """
        Initialize the EmployeeSalaryCalculator with salary.
        Parameters:
            salary (float): The gross salary of the employee.
        """
        self.salary = salary
    def calculate_tax(self):
        """
        Calculate tax based on 20% of salary.
        Returns:
            float: Calculated tax amount.
        """
        return self.salary * 0.2
    def calculate_net_salary(self):
        """
        Calculate net salary after deducting tax.
        Returns:
            float: Net salary after tax deduction.
        """
        tax = self.calculate_tax()
        return self.salary - tax
# Create object
employee = EmployeeSalaryCalculator(50000)
# Calculate and print net salary
print(employee.calculate_net_salary())
```

```
PS C:\Users\SONY REDDY\OneDrive\Desktop\AI ASSISTANT CODING> & "C:/Users/SONY REDDY/AppData/Local/Programs/Python/Python313/python.exe"
SSISTANT CODING/Ass-13.py"
40000.0
40000.0
```

Task 6 (Optimizing Search Logic)

• Task: Refactor inefficient linear searches using appropriate data structures.

• Focus Areas:

o Time complexity

o Data structure choice

Legacy Code:

users = ["admin", "guest", "editor", "viewer"]

name = input("Enter username: ")

found = False

for u in users:

if u == name:

found = True

print("Access Granted" if found else "Access Denied")

Expected Outcome:Use of sets or dictionaries with complexity justification

```
Ass-13.py > ...
users = ["admin", "guest", "editor", "viewer"]
name = input("Enter username: ")
found = False
for u in users:
    if u == name:
        found = True
print("Access Granted" if found else "Access Denied")
#Refactor the linear search code by replacing the list with a more efficient data structure like a set or dictionary.
#Improve time complexity, avoid manual looping, and keep the same output behavior
"""
User Authentication Module
This module optimizes username search by using a set
to improve lookup time complexity.
"""
# Using a set for faster membership testing (Average O(1) time complexity)
users = {"admin", "guest", "editor", "viewer"}
name = input("Enter username: ")
# Direct membership check (no manual loop needed)
if name in users:
    print("Access Granted")
else:
    print("Access Denied")
```

```
PROBLEMS   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS                                   Python

PS C:\Users\SONY REDDY\OneDrive\Desktop\AI ASSISTANT CODING> & "C:/Users/SONY REDDY/AppData/Local/Programs/Python/Python313/python.exe" "c:/Users/SONY REDDY/OneDriv
SSISTANT CODING/Ass-13.py"
Enter username: admin
Access Granted
Enter username: guest
Access Granted
```

Task 7 – Refactoring the Library Management System

Problem Statement

You are provided with a poorly structured Library Management script that:

• Contains repeated conditional logic

• Does not use reusable functions

• Lacks documentation

• Uses print-based procedural execution

• Does not follow modular programming principles

Your task is to refactor the code into a proper format

1. Create a module library.py with functions:

o add_book(title, author, isbn)

o remove_book(isbn)

o search_book(isbn)

2. Insert triple quotes under each function and let Copilot complete the docstrings.

3. Generate documentation in the terminal.

4. Export the documentation in HTML format.

5. Open the file in a browser.

Given Code

```
# Library Management System (Unstructured Version)
# This code needs refactoring into a proper module with documentation.
library_db = {}
# Adding first book
title = "Python Basics"
author = "John Doe"
isbn = "101"
if isbn not in library_db:
library_db[isbn] = {"title": title, "author": author}
print("Book added successfully.")
else:
print("Book already exists.")
# Adding second book (duplicate logic)
title = "AI Fundamentals"
author = "Jane Smith"
isbn = "102"
if isbn not in library_db:
library_db[isbn] = {"title": title, "author": author}
print("Book added successfully.")
else:
print("Book already exists.")
# Searching book (repeated logic structure)
isbn = "101"
if isbn in library_db:
print("Book Found:", library_db[isbn])
else:
print("Book not found.")
# Removing book (again repeated pattern)
isbn = "101"
```

if isbn in library_db:

del library_db[isbn]

print("Book removed successfully.")

else:

print("Book not found.")

Searching again

isbn = "101"

if isbn in library_db:

print("Book Found:", library_db[isbn])

else:

print("Book not found.")

Ass-13.py > ...

```python
# Library Management System (Unstructured Version)
# This code needs refactoring into a proper module with documentation.
library_db = {}
# Adding first book
title = "Python Basics"
author = "John Doe"
isbn = "101"
if isbn not in library_db:
    library_db[isbn] = {"title": title, "author": author}
    print("Book added successfully.")
else:
    print("Book already exists.")
# Adding second book (duplicate logic)
title = "AI Fundamentals"
author = "Jane Smith"
isbn = "102"
if isbn not in library_db:
    library_db[isbn] = {"title": title, "author": author}
    print("Book added successfully.")
else:
    print("Book already exists.")
# Searching book (repeated logic structure)
isbn = "101"
if isbn in library_db:
    print("Book Found:", library_db[isbn])
else:
    print("Book not found.")
# Removing book (again repeated pattern)
isbn = "101"
if isbn in library_db:
    del library_db[isbn]
    print("Book removed successfully.")
else:
    print("Book not found.")
# Searching again
isbn = "101"
if isbn in library_db:
    print("Book Found:", library_db[isbn])
else:
    print("Book not found.")
#Refactor the given unstructured code into a clean, modular design with reusable functions, proper docstrings,
# and no repeated logic while maintaining the same functionality.
```

```python
Ass-13.py > ...
Library Management Module
This module provides reusable functions to manage
a simple in-memory library database.
"""
library_db = {}
def add_book(title, author, isbn):
    """
    Add a new book to the library database.
    Parameters:
        title (str): Title of the book.
        author (str): Author of the book.
        isbn (str): Unique ISBN identifier.
    Returns:
        str: Success or failure message.
    """
    if isbn not in library_db:
        library_db[isbn] = {"title": title, "author": author}
        return "Book added successfully."
    return "Book already exists."
def remove_book(isbn):
    """
    Remove a book from the library database.
    Parameters:
        isbn (str): Unique ISBN identifier.

    Returns:
        str: Success or failure message.
    """
    if isbn in library_db:
        del library_db[isbn]
        return "Book removed successfully."
    return "Book not found."
def search_book(isbn):
    """
    Search for a book in the library database.
    Parameters:
        isbn (str): Unique ISBN identifier.

    Returns:
        dict or str: Book details if found, otherwise message.
    """
    if isbn in library_db:
        return library_db[isbn]
    return "Book not found."
```

```python
Ass-13.py > ...
def search_book(isbn):
        dict or str: Book details if found, otherwise message.
    """
    if isbn in library_db:
        return library_db[isbn]
    return "Book not found."
# Example Usage
if __name__ == "__main__":
    print(add_book("Python Basics", "John Doe", "101"))
    print(add_book("AI Fundamentals", "Jane Smith", "102"))
    print(search_book("101"))
    print(remove_book("101"))
    print(search_book("101"))
```

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS                                                    Python
Book removed successfully.
Book not found.
Book added successfully. …
PS C:\Users\SONY REDDY\OneDrive\Desktop\AI ASSISTANT CODING> & "C:/Users/SONY REDDY/AppData/Local/Programs/Python/Python313/python.exe" "c:/Users/SONY REDDY/OneDrive/Desktop/AI ASSISTANT CODING/
Book added successfully.
Book added successfully.
Book Found: {'title': 'Python Basics', 'author': 'John Doe'}
Book removed successfully.
Book not found.
Book added successfully.
Book added successfully.
{'title': 'Python Basics', 'author': 'John Doe'}
Book removed successfully.
Book not found.
```

Task 8– Fibonacci Generator Write a program to generate Fibonacci series up to n.

The initial code has:

- Global variables.
- Inefficient loop.
- No functions or modularity.

Task for Students:

- Refactor into a clean reusable function (generate_fibonacci).
- Add docstrings and test cases.
- Compare AI-refactored vs original.

Bad Code Version:

fibonacci bad version

n=int(input("Enter limit: "))

a=0

b=1

print(a)

print(b)

for i in range(2,n):

c=a+b

print(c)

a=b

b=c

Ass-13.py

```python
# fibonacci bad version
n = int(input("Enter limit: "))
a = 0
b = 1
print(a)
print(b)
for i in range(2, n):
    c = a + b
    print(c)
    a = b
    b = c
#Refactor the Fibonacci script into a clean, reusable function generate_fibonacci(n)
# with proper docstrings, no global variables, added test cases, and improved modularity while keeping the same output.
"""
Fibonacci Generator Module
This module provides a reusable function to generate
the Fibonacci series up to n terms.
"""
def generate_fibonacci(n):
    """
    Generate Fibonacci series up to n terms.
    Parameters:
        n (int): Number of terms to generate.
    Returns:
        list: A list containing Fibonacci series up to n terms.
    """
    if n <= 0:
        return []
    if n == 1:
        return [0]
    fibonacci_series = [0, 1]
    for _ in range(2, n):
        fibonacci_series.append(
            fibonacci_series[-1] + fibonacci_series[-2]
        )
    return fibonacci_series
# Test Cases
if __name__ == "__main__":
    print("Fibonacci Series (5 terms):", generate_fibonacci(5))
    print("Fibonacci Series (1 term):", generate_fibonacci(1))
    print("Fibonacci Series (0 terms):", generate_fibonacci(0))
```

```
PS C:\Users\SONY REDDY\OneDrive\Desktop\AI ASSISTANT CODING> & "C:/Users/SONY REDDY/AppData/Local/Programs/Python/Python313/python.exe" "c:/Users/SONY REDDY/D
esktop/AI ASSISTANT CODING/Ass-13.py"
Enter limit: 5
0
1
1
2
3
Fibonacci Series (5 terms): [0, 1, 1, 2, 3]
Fibonacci Series (1 term): [0]
Fibonacci Series (0 terms): []
```

Task 9 – Twin Primes Checker

Twin primes are pairs of primes that differ by 2 (e.g., 11 and 13, 17 and 19).The initial code has:

• Inefficient prime checking.

• No functions.

• Hardcoded inputs.

Task for Students:

• Refactor into is_prime(n) and is_twin_prime(p1, p2).

• Add docstrings and optimize.

• Generate a list of twin primes in a given range using AI.Bad Code Version:

```
# twin primes bad version
a=11
b=13
fa=0
for i in range(2,a):
if a%i==0:
fa=1
fb=0
for i in range(2,b):
if b%i==0:
fb=1
if fa==0 and fb==0 and abs(a-b)==2:
print("Twin Primes")
else:
print("Not Twin Primes")
```

Ass-13.py > ...

```python
# twin primes bad version
a=11
b=13
fa=0
for i in range(2,a):
    if a%i==0:
        fa=1
fb=0
for i in range(2,b):
    if b%i==0:
        fb=1
if fa==0 and fb==0 and abs(a-b)==2:
    print("Twin Primes")
else:
    print("Not Twin Primes")
#Refactor the twin prime checker by creating functions is_prime(n) and is_twin_prime(p1, p2), add docstrings,
# optimize the prime check, remove hardcoded logic, and generate twin primes within a given range.
import math
def is_prime(n):
    """
    Check whether a number is prime using optimized logic.
    Args:
        n (int): Number to check
    Returns:
        bool: True if prime, otherwise False
    """
    if n < 2:
        return False
    for i in range(2, int(math.sqrt(n)) + 1):
        if n % i == 0:
            return False
    return True
def is_twin_prime(p1, p2):
```

Ass-13.py > ...

```python
def is_twin_prime(p1, p2):
    """
    Check whether two numbers are twin primes.
    Args:
        p1 (int): First number
        p2 (int): Second number
    Returns:
        bool: True if twin primes, otherwise False
    """
    return is_prime(p1) and is_prime(p2) and abs(p1 - p2) == 2
def generate_twin_primes(start, end):
    """
    Generate all twin prime pairs within a given range.
    Args:
        start (int): Start of range
        end (int): End of range
    Returns:
        list: List of twin prime tuples
    """
    twins = []
    for num in range(start, end - 1):
        if is_twin_prime(num, num + 2):
            twins.append((num, num + 2))
    return twins
a, b = 11, 13
if is_twin_prime(a, b):
    print("Twin Primes")
else:
    print("Not Twin Primes")
print("Twin primes between 1 and 50:", generate_twin_primes(1, 50))
```

```
PS C:\Users\SONY REDDY\OneDrive\Desktop\AI ASSISTANT CODING> & "C:/Users/SONY REDDY/AppData/Local/Programs/Python/Python313/python.exe" "c:/Users/SONY R
SSISTANT CODING/Ass-13.py"
Twin Primes
Twin Primes
Twin primes between 1 and 50: [(3, 5), (5, 7), (11, 13), (17, 19), (29, 31), (41, 43)]
```

Task 10 – Refactoring the Chinese Zodiac Program

Objective

Refactor the given poorly structured Python script into a clean, modular, and reusable implementation.

The current program reads a year from the user and prints the corresponding

Chinese Zodiac sign. However, the implementation contains repetitive

conditional logic, lacks modular design, and does not follow clean coding principles.

Your task is to refactor the code to improve readability, maintainability, and structure.

Chinese Zodiac Cycle (Repeats Every 12 Years)

1. Rat

2. Ox

3. Tiger

4. Rabbit

5. Dragon

6. Snake

7. Horse

8. Goat (Sheep)

9. Monkey

10. Rooster

11. Dog

12. Pig

```
# Chinese Zodiac Program (Unstructured Version)

# This code needs refactoring.

year = int(input("Enter a year: "))

if year % 12 == 0:

print("Monkey")

elif year % 12 == 1:

print("Rooster")

elif year % 12 == 2:

print("Dog")

elif year % 12 == 3:

print("Pig")

elif year % 12 == 4:

print("Rat")

elif year % 12 == 5:

print("Ox")

elif year % 12 == 6:
```

print("Tiger")

elif year % 12 == 7:

print("Rabbit")

elif year % 12 == 8:

print("Dragon")

elif year % 12 == 9:

print("Snake")

elif year % 12 == 10:

print("Horse")

elif year % 12 == 11:

print("Goat")

You must:

1. Create a reusable function: get_zodiac(year)

2. Replace the if-elif chain with a cleaner structure (e.g., list or dictionary).

3. Add proper docstrings.

4. Separate input handling from logic.

5. Improve readability and maintainability.

6. Ensure output remains correct.

Ass-13.py > ...

```python
# Chinese Zodiac Program (Unstructured Version)
# This code needs refactoring.
year = int(input("Enter a year: "))
if year % 12 == 0:
    print("Monkey")
elif year % 12 == 1:
    print("Rooster")
elif year % 12 == 2:
    print("Dog")
elif year % 12 == 3:
    print("Pig")
elif year % 12 == 4:
    print("Rat")
elif year % 12 == 5:
    print("Ox")
elif year % 12 == 6:
    print("Tiger")
elif year % 12 == 7:
    print("Rabbit")
elif year % 12 == 8:
    print("Dragon")
elif year % 12 == 9:
    print("Snake")
elif year % 12 == 10:
    print("Horse")
elif year % 12 == 11:
    print("Goat")
#Refactor the Chinese Zodiac program into a clean, modular version by creating a reusable get_zodiac(year) function,
# using a list or dictionary instead of if-elif statements, adding docstrings, separating input from logic, and maintaining correct output.
"""
Chinese Zodiac Module
This module provides a reusable function to determine
the Chinese Zodiac sign for a given year.
"""
def get_zodiac(year):
    """
```

```python
def get_zodiac(year):
    """
    Determine the Chinese Zodiac sign for a given year.
    Parameters:
        year (int): The year for which the zodiac sign is required.
    Returns:
        str: The corresponding Chinese Zodiac sign.
    """
    zodiac_cycle = [
        "Monkey", "Rooster", "Dog", "Pig",
        "Rat", "Ox", "Tiger", "Rabbit",
        "Dragon", "Snake", "Horse", "Goat"
    ]
    return zodiac_cycle[year % 12]
if __name__ == "__main__":
    year_input = int(input("Enter a year: "))
    print(get_zodiac(year_input))
```

```
PS C:\Users\SONY REDDY\OneDrive\Desktop\AI ASSISTANT CODING> & "C:/Users/SONY REDDY/AppData/Local/Programs/Python/Python313/python.exe" "c:/Users/SONY REDDY/OneDriv
SSISTANT CODING/Ass-13.py"
Enter a year: 2025
Snake
Enter a year: 2020
Rat
```

Task 11 – Refactoring the Harshad (Niven) Number Checker

Refactor the given poorly structured Python script into a clean, modular, and reusable implementation.

A Harshad (Niven) number is a number that is divisible by the sum of its digits.

For example:

• 18 → 1 + 8 = 9 → 18 ÷ 9 = 2 ⍰ (Harshad Number)

• 19 → 1 + 9 = 10 → 19 ÷ 10 ≠ integer ⍰ (Not Harshad)

Problem Statement

The current implementation:

• Mixes logic and input handling

• Uses redundant variables

• Does not use reusable functions properly

• Returns print statements instead of boolean values

• Lacks documentation

You must refactor the code to follow clean coding principles.

Harshad Number Checker (Unstructured Version)

num = int(input("Enter a number: "))

temp = num

sum_digits = 0

while temp > 0:

digit = temp % 10

sum_digits = sum_digits + digit

temp = temp // 10

if sum_digits != 0:

if num % sum_digits == 0:

print("True")

else:

print("False")

else:

print("False")

You must:

1. Create a reusable function: is_harshad(number)

2. The function must:

o Accept an integer parameter.

o Return True if the number is divisible by the sum of its digits.

o Return False otherwise.

3. Separate user input from core logic.

4. Add proper docstrings.

5. Improve readability and maintainability.

6. Ensure the program handles edge cases (e.g., 0, negative numbers).

```
Ass-13.py > ...
# Harshad Number Checker (Unstructured Version)
num = int(input("Enter a number: "))
temp = num
sum_digits = 0
while temp > 0:
    digit = temp % 10
    sum_digits = sum_digits + digit
    temp = temp // 10
if sum_digits != 0:
    if num % sum_digits == 0:
        print("True")
    else:
        print("False")
else:
    print("False")
#Refactor the Harshad number checker by creating a reusable function is_harshad(number) that returns True or False,
# separate input from logic, add docstrings, improve readability, and handle edge cases like zero and negative numbers.
def is_harshad(number):
    """
    Determine whether a number is a Harshad (Niven) number.
    A Harshad number is divisible by the sum of its digits.
    Args:
        number (int): Number to check
    Returns:
        bool: True if Harshad number, otherwise False
    """
    # Handle edge cases
    if number <= 0:
        return False
    # Calculate sum of digits
    digit_sum = sum(int(digit) for digit in str(number))
    # Prevent division by zero
    if digit_sum == 0:
        return False
    return number % digit_sum == 0
# Input handling separated from logic
num = int(input("Enter a number: "))
# Display result
print(is_harshad(num))
```

```
PS C:\Users\SONY REDDY\OneDrive\Desktop\AI ASSISTANT CODING> & "C:/Users/SONY REDDY/AppData/Local/Programs/Python/Python313/python.exe" "c:/Users/SONY REDDY/On
SSISTANT CODING/Ass-13.py"
Enter a number: 20
True
Enter a number: 2025
True
```

Task 12 – Refactoring the Factorial Trailing Zeros Program

Refactor the given poorly structured Python script into a clean, modular, and efficient implementation.

The program calculates the number of trailing zeros in n! (factorial of n).

Problem Statement

The current implementation:

• Calculates the full factorial (inefficient for large n)

• Mixes input handling with business logic

• Uses print statements instead of return values

• Lacks modular structure and documentation

You must refactor the code to improve efficiency, readability, and maintainability.

```
# Factorial Trailing Zeros (Unstructured Version)
n = int(input("Enter a number: "))
fact = 1
i = 1
while i <= n:
fact = fact * i
i = i + 1
count = 0
while fact % 10 == 0:
count = count + 1
fact = fact // 10
print("Trailing zeros:", count)
```

You must:

1. Create a reusable function: count_trailing_zeros(n)

2. The function must:

o Accept a non-negative integer n.

o Return the number of trailing zeros in n!.

3. Do NOT compute the full factorial.

4. Use an optimized mathematical approach (count multiples of 5).

5. Add proper docstrings.

6. Separate user interaction from core logic.

7. Handle edge cases (e.g., negative numbers, zero).

```python
Ass-13.py > ...
# Factorial Trailing Zeros (Unstructured Version)
n = int(input("Enter a number: "))
fact = 1
i = 1
while i <= n:
    fact = fact * i
    i = i + 1
count = 0
while fact % 10 == 0:
    count = count + 1
    fact = fact // 10
print("Trailing zeros:", count)
#Refactor the trailing zeros program into a clean, reusable count_trailing_zeros(n) function using the optimized multiples-of-5 approach,
# with proper docstrings, separated input handling, edge case support, and correct output.
"""
Factorial Trailing Zeros Module
This module provides an efficient function to calculate
the number of trailing zeros in n! without computing
the full factorial.
"""
def count_trailing_zeros(n):
    """
    Calculate the number of trailing zeros in n! (factorial of n).
    This function uses the optimized mathematical approach
    of counting multiples of 5, since trailing zeros are
    created by factors of 10 (2 × 5), and 5 is the limiting factor.
    Parameters:
        n (int): A non-negative integer.
    Returns:
        int: Number of trailing zeros in n!.
    Raises:
        ValueError: If n is negative.
    """
    if n < 0:
        raise ValueError("Input must be a non-negative integer.")
    count = 0
```

```python
    if n < 0:
        raise ValueError("Input must be a non-negative integer.")
    count = 0
    divisor = 5
    while n // divisor > 0:
        count += n // divisor
        divisor *= 5
    return count
if __name__ == "__main__":
    try:
        number = int(input("Enter a number: "))
        result = count_trailing_zeros(number)
        print("Trailing zeros:", result)
    except ValueError as e:
        print("Error:", e)
```

```
PS C:\Users\SONY REDDY\OneDrive\Desktop\AI ASSISTANT CODING> & "C:/Users/SONY REDDY/AppData/Local/Programs/Python/Python313/python.exe" "c:/Users/SONY RE
esktop/AI ASSISTANT CODING/Ass-13.py"
Enter a number: 45
Trailing zeros: 10
Enter a number: 2025
Trailing zeros: 505
```

Task 13 (Collatz Sequence Generator – Test Case Design)

- Function: Generate Collatz sequence until reaching 1.
- Test Cases to Design:
- Normal: 6 → [6,3,10,5,16,8,4,2,1]
- Edge: 1 → [1]
- Negative: -5
- Large: 27 (well-known long sequence)

• Requirement: Validate correctness with pytest.

Explanation:

We need to write a function that:

• Takes an integer n as input.

• Generates the Collatz sequence (also called the 3n+1 sequence).

• The rules are:

o If n is even → next = n / 2.

o If n is odd → next = 3n + 1.

• Repeat until we reach 1.

• Return the full sequence as a list.

Example

Input: 6

Steps:

• 6 (even → 6/2 = 3)

• 3 (odd → 3*3+1 = 10)

• 10 (even → 10/2 = 5)

• 5 (odd → 3*5+1 = 16)

• 16 (even → 16/2 = 8)

• 8 (even → 8/2 = 4)

• 4 (even → 4/2 = 2)

• 2 (even → 2/2 = 1)

Output:

[6, 3, 10, 5, 16, 8, 4, 2, 1]

```python
collatz.py > {} collatz_sequence
# Collatz Sequence Generator (To Be Implemented)
# Rules:
# If n is even → n = n / 2
# If n is odd → n = 3n + 1
# Repeat until n becomes 1
# Return full sequence as list
#Create a reusable function collatz_sequence(n) that generates the Collatz sequence until 1, handles edge and
# negative cases, returns the full list, and validate correctness using pytest.
"""
Collatz Sequence Module
This module provides a function to generate the
Collatz (3n + 1) sequence for a given integer.
"""
def collatz_sequence(n):
    """
    Generate the Collatz sequence starting from n until reaching 1.
    Parameters:
        n (int): A positive integer.
    Returns:
        list: The Collatz sequence ending in 1.
    Raises:
        ValueError: If n is less than 1.
    """
    if n < 1:
        raise ValueError("Input must be a positive integer.")
    sequence = [n]
    while n != 1:
        if n % 2 == 0:
            n = n // 2
        else:
            n = 3 * n + 1
        sequence.append(n)
    return sequence
    import pytest
from collatz import collatz_sequence
def test_normal_case():
    assert collatz_sequence(6) == [6, 3, 10, 5, 16, 8, 4, 2, 1]
def test_edge_case():
```

```python
from collatz import collatz_sequence
def test_normal_case():
    assert collatz_sequence(6) == [6, 3, 10, 5, 16, 8, 4, 2, 1]
def test_edge_case():
    assert collatz_sequence(1) == [1]
def test_negative_case():
    with pytest.raises(ValueError):
        collatz_sequence(-5)
def test_large_case():
    result = collatz_sequence(27)
    assert result[0] == 27
    assert result[-1] == 1
```

```
PS C:\Users\HP\Desktop\AI-Lab> python -m pytest  rectangle_refactored.py
========================================================================== test session starts ==========================================================
platform win32 -- Python 3.13.7, pytest-9.0.2, pluggy-1.6.0
rootdir: C:\Users\HP\Desktop\AI-Lab
plugins: anyio-4.10.0
collected 4 items

rectangle_refactored.py ....

========================================================================== 4 passed in 0.07s ==========================================================
```

Task 14 (Lucas Number Sequence – Test Case Design)

• Function: Generate Lucas sequence up to n terms.

(Starts with 2,1, then Fn = Fn-1 + Fn-2)

• Test Cases to Design:

• Normal: 5 → [2, 1, 3, 4, 7]

• Edge: 1 → [2]

• Negative: -5 → Error

• Large: 10 (last element = 76).

• Requirement: Validate correctness with pytest.

```python
test_lucas.py > ...
# Lucas Number Sequence (To Be Implemented)
# Rules:
# - Starts with 2, 1
# - Next term = sum of previous two terms
# - Generate sequence up to n terms
# - Validate using pytest
#Create a reusable function generate_lucas(n) that returns the Lucas sequence up to n terms,
# handles edge and negative cases properly, and validate correctness using pytest.
"""
Lucas Sequence Module
This module provides a function to generate
the Lucas number sequence up to n terms.
"""
def generate_lucas(n):
    """
    Generate the Lucas sequence up to n terms.
    Parameters:
        n (int): Number of terms to generate.
    Returns:
        list: Lucas sequence up to n terms.
    Raises:
        ValueError: If n is negative.
    """
    if n < 0:
        raise ValueError("Input must be a non-negative integer.")
    if n == 0:
        return []
    if n == 1:
        return [2]
    sequence = [2, 1]
```

```python
    sequence = [2, 1]
    for _ in range(2, n):
        sequence.append(sequence[-1] + sequence[-2])
    return sequence
    import pytest
from lucas import generate_lucas
def test_normal_case():
    assert generate_lucas(5) == [2, 1, 3, 4, 7]
def test_edge_case():
    assert generate_lucas(1) == [2]
def test_negative_case():
    with pytest.raises(ValueError):
        generate_lucas(-5)
def test_large_case():
    result = generate_lucas(10)
    assert result[-1] == 76
```

```
PS C:\Users\HP\Desktop\AI-Lab> python -m pytest  rectangle_refactored.py
========================================================================== test session starts ==========================================================================
platform win32 -- Python 3.13.7, pytest-9.0.2, pluggy-1.6.0
rootdir: C:\Users\HP\Desktop\AI-Lab
plugins: anyio-4.10.0
collected 4 items

rectangle_refactored.py ....

========================================================================== 4 passed in 0.07s ==========================================================================
```

Task 15 (Vowel & Consonant Counter – Test Case Design)

• Function: Count vowels and consonants in string.

• Test Cases to Design:

• Normal: "hello" → (2,3)

• Edge: "" → (0,0)

• Only vowels: "aeiou" → (5,0)

Large: Long text

• Requirement: Validate correctness with pytest.

rectangle_refactored.py > ...

```python
# Vowel & Consonant Counter (To Be Implemented)
# Requirements:
# - Count vowels and consonants in a string
# - Return result as a tuple (vowels, consonants)
# - Validate using pytest
#Create a reusable function count_vowels_consonants(text) that returns the number of vowels and
# consonants in a string, handles edge cases, and validate correctness using pytest.
"""
Vowel and Consonant Counter Module
This module provides a function to count vowels
and consonants in a given string.
"""
def count_vowels_consonants(text):
    """
    Count vowels and consonants in a string.

    Parameters:
        text (str): Input string.
    Returns:
        tuple: (vowel_count, consonant_count)
    """
    vowels = "aeiouAEIOU"
    vowel_count = 0
    consonant_count = 0
    for char in text:
        if char.isalpha():
            if char in vowels:
                vowel_count += 1
            else:
                consonant_count += 1
    return (vowel_count, consonant_count)
    import pytest
from vowel_counter import count_vowels_consonants
def test_normal_case():
    assert count_vowels_consonants("hello") == (2, 3)
def test_edge_case():
    assert count_vowels_consonants("") == (0, 0)
```

```python
def test_edge_case():
    assert count_vowels_consonants("") == (0, 0)
def test_only_vowels():
    assert count_vowels_consonants("aeiou") == (5, 0)
def test_large_input():
    text = "This is a long text used for testing purposes"
    vowels, consonants = count_vowels_consonants(text)
    assert vowels > 0
    assert consonants > 0
```

```
PS C:\Users\HP\Desktop\AI-Lab> python -m pytest  rectangle_refactored.py
========================================================================== test session starts ==========================================================================
platform win32 -- Python 3.13.7, pytest-9.0.2, pluggy-1.6.0
rootdir: C:\Users\HP\Desktop\AI-Lab
plugins: anyio-4.10.0
collected 4 items

rectangle_refactored.py ....

========================================================================== 4 passed in 0.07s ==========================================================================
```